

Variables as input to functions

When a variable is used as input to a function its value is copied into a new local variable. Therefore, changing the value of this new local variable inside the function doesn't change the value of the original variable.

1 Changing values inside a variable

"func3" uses input "y", which is a local variable. It prints the value of "y", then changes that value to "bread" and prints the new value.

```
>>> def func3(y):
    print(y)
    y = 'bread'
    print(y)
>>> z = 'butter'
>>> func3(z)
butter
bread
```

Annotations for the first code block:

- Arrow from `y` in `func3(y):` to `print(y)`: "y" contains the value passed to it when "func3" is called
- Arrow from `y` in `func3(y):` to `y = 'bread'`: Here "y" contains "bread"
- Arrow from `z` in `func3(z)` to `func3(z)`: This creates a global variable called "z"
- Arrow from `func3(z)` to the output `butter`: The input "y" now contains the value of "z" passed to it when "func3" is called

2 Print variable

Printing the value of "z" after calling "func3" shows it hasn't changed. Calling "func3" copies the value in "z" ("butter") into local variable "y", but "z" is left unchanged.

```
>>> print(z)
butter
```

Annotations for the second code block:

- Arrow from `print(z)` to `print(z)`: Prints the value in global variable "z" after "func3" has finished running
- Arrow from `butter` to the text below: Local variable "y" of "func3" holds a copy of the value in "z". Although "y" has been changed to "bread", the value in global variable "z" isn't affected and is still "butter"

Masking a global variable

A global variable can't be changed by a function. A function trying to change a global variable actually creates a local variable with the same name. It covers up, or "masks", the global variable with a local version.

1 Changing a global variable

Global variable "c" is given the value 12345. "func4" gives "c" the value 555 and prints it out. It looks like our global variable "c" has been changed.

```
>>> c = 12345
>>> def func4():
    c = 555
    print(c)
>>> func4()
555
```

Annotations for the third code block:

- Arrow from `c = 12345` to `c = 12345`: Initial value in global variable "c"
- Arrow from `func4()` to `print(c)`: Prints the value of "c" inside "func4"

2 Print variable

If we print "c" from outside the function, we see that "c" hasn't changed at all. "func4" only prints the value of its new local variable—also called "c".

```
>>> print(c)
12345
```

Annotation for the fourth code block:

- Arrow from `print(c)` to the output `12345`: The value in global variable "c" hasn't been changed



EXPERT TIPS

Calling functions

There are two different ways of calling functions.

function(a)

In Python, items of data are called "objects". Some functions are called by passing them the data object ("a").

a.function()

Other functions are called by adding their name at the end of the data object ("a") after a period.